```
#Practical No 8
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


# Construct expression tree from prefix expression
def construct_tree(prefix):
    stack = []

    for ch in reversed(prefix.split()):
        node = Node(ch)

        if ch in "+-*/":
            node.left = stack.pop()
            node.right = stack.pop()

        stack.append(node)

    return stack.pop()


# Non-recursive postorder traversal
def postorder(root):
    if root is None:
        return

    stack1 = [root]
    stack2 = []

    while stack1:
        node = stack1.pop()
        stack2.append(node)

        if node.left:
            stack1.append(node.left)
        if node.right:
            stack1.append(node.right)

    print("Postorder Traversal:", end=" ")
    while stack2:
        print(stack2.pop().data, end=" ")
```

```
    print()


# Delete the entire tree
def delete_tree(root):
    if root is None:
        return None

    stack = [root]

    while stack:
        node = stack.pop()

        if node.left:
            stack.append(node.left)
        if node.right:
            stack.append(node.right)

        node.left = None
        node.right = None

    return None


# Main program
prefix = input("Enter prefix expression (space separated): ")

root = construct_tree(prefix)

print("Expression tree constructed successfully.")
postorder(root)

root = delete_tree(root)
print("Expression tree deleted successfully.")
```

Sample Output :

```
Enter prefix expression (space separated): + * A B - C D
Expression tree constructed successfully.
Postorder Traversal: A B * C D - +
Expression tree deleted successfully.
```